

AI-DRIVEN DEVELOPMENT 2026

VSCode × Claude Code 実践ハンズオン

プログラミング未経験でも大丈夫です。AIに指示を出してウェブアプリを開発する体験を通じ、業務課題を自分の手で解決できる力を身につけます。

 4時間（13:00 - 17:00） 講義 3 : 実習 7 VSCode + Claude Code

タイムテーブル

時間	区分	形式	内容
13:00 - 13:20	Session 1	座学	AI駆動開発入門 —— ゴール共有とAI駆動開発の全体像
13:20 - 13:50	Session 2	実践	環境構築ハンズオン —— Node.js・Claude Codeのセットアップ
13:50 - 14:00	休憩	-	
14:00 - 14:50	Session 3	実践	ミニ業務アプリ開発 —— チェックリストアプリでClaude Codeに慣れる
14:50 - 16:10	Session 4	実践	PDF文書比較ツール —— サーバー+AI APIを使った本格アプリの構築
16:10 - 16:20	休憩	-	
16:20 - 17:00	Session 5	座学	生成AI資格ガイド —— 生成AIパスポートと資格ロードマップ

セッション概要

SESSION 01

AI駆動開発入門

AIに指示を出してアプリを作る「AI駆動開発」の全体像を把握します。従来の開発との違い、

SESSION 02

環境構築ハンズオン

Node.jsとClaude Codeをインストールし、ターミナルの基本操作を覚えます。VSCodeは事前イ

Claude Codeでできること、本日の到達点を共有します。

20分 座学

Session 1 を開く →

インストール済みの前提で進めます。

30分 実践

Session 2 を開く →

SESSION 03

ミニ業務アプリ開発

Claude Codeに要件を伝えて業務アプリを作る初体験です。設備点検チェックリストの開発を通じ、プロンプトの書き方と改善サイクルを身につけます。

50分 実践

Session 3 を開く →

SESSION 04

PDF文書比較ツール

講師デモを見た後、自分のプロンプトでPDF比較ツールをゼロから開発します。プロンプト設計力が試される実践セッションです。

80分 実践

Session 4 を開く →

SESSION 05

生成AI 資格ガイド

生成AIパスポートを起点に、キャリア方向に合った資格を7方向15資格から選ぶロードマップをご紹介します。

40分 座学

Session 5 を開く →

使用ツール



VSCode
コードエディタ



Claude Code
AI駆動開発ツール



Node.js
JavaScript実行環境



ブラウザ
Chrome推奨

事前準備

ノートPC

Windows または Mac をご用意ください。インターネットに接続できる環境が必要です。

組織招待メールの承認

研修事務局から届く「ClaudeCodeTutorial」への招待メールを事前に承認し、アカウントを作成しておいてください。

VSCode のインストール

code.visualstudio.com からダウンロードしてインストールしておくと、当日スムーズに進められます。

心構え

プログラミング経験は一切不要です。日本語でAIに指示を出し、動くアプリを持ち帰る4時間になります。

[Session 1](#) [Session 2](#) [Session 3](#) [Session 4](#) [Session 5](#)

VSCode x Claude Code Hands-on Training Program

SESSION 01

AI駆動開発入門

コードを1行も書かずにウェブアプリを作る時代が来ています。AIに日本語で指示を出し、設計からコーディングまでを任せる「AI駆動開発」の全体像をつかんでいただきます。

🕒 13:00 - 13:20 (20分)

📖 座学

INTRODUCTION

本日の到達ゴール

この研修を終えたとき、皆さんは「自分の業務課題をウェブアプリとして形にし、社内で共有するイメージ」を持てるようになっています。

プログラミング経験は問いません。日本語でAIに指示を出せば、それが開発の出発点です。

4時間の流れ

前半では環境を整え、チェックリストアプリでClaude Codeの操作感を掴みます。後半では本日の最終成果物であるPDF文書比較ツールをセットアップし、Claude Codeでカスタマイズします。最後に、作ったアプリを社内で活用するための道筋もお伝えします。

前半
環境構築 + ミニ業務アプリ



後半
PDF文書比較ツール

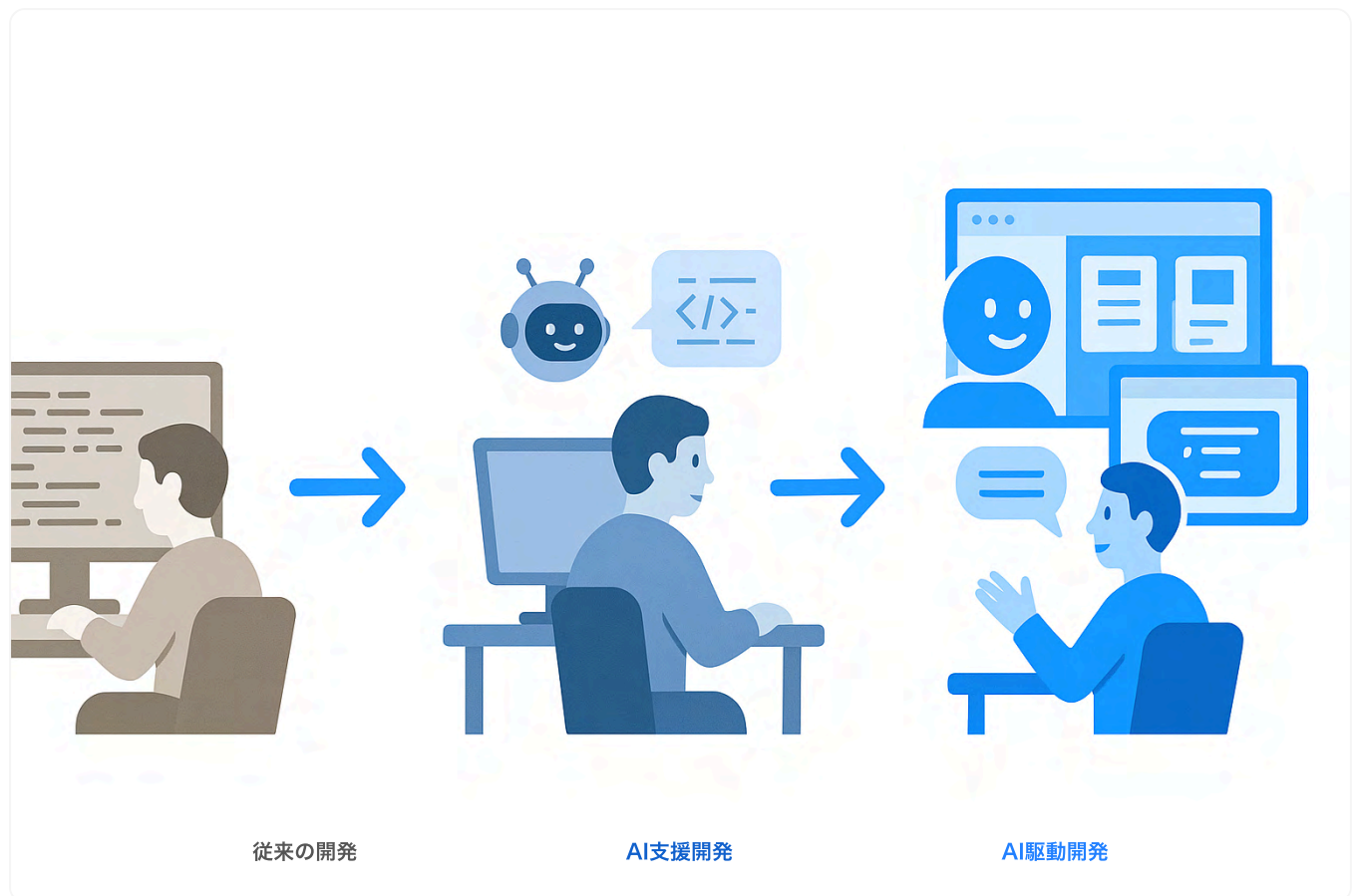


まとめ
振り返り + 次のステップ

AI駆動開発とは何か

3つの開発スタイル

ソフトウェアの作り方は、この数年で3段階の変化を経てきました。



STAGE 01

従来の開発

プログラマーがコードを1行ずつ書く。設計書を作り、実装し、テストする。すべて人間の手作業です。

- 専門スキルが必要
- 時間がかかる
- 品質は書く人の力量に依存

STAGE 02

AI支援開発

GitHub CopilotのようなツールがコードをAI補完する。タイピングは減るが、開発者がコードの方向性を制御する点は変わりません。

- コード補完で生産性向上
- プログラミング知識は必要
- あくまで「補助」の立場

STAGE 03 — 今回の研修で体験するもの

AI駆動開発 (Vibe Coding)

人間は「何を作りたいか」を日本語で伝え、AIが設計からコーディング、修正まで一貫して行います。プログラミング言語を知らなくても、動くアプリが手に入ります。

- 日本語の指示（プロンプト）が開発の起点
- AIがファイル作成、コード生成、デバッグまで実行
- 人間は「レビュアー」として品質を判断する役割
- 開発のスピードが桁違いに速い

Claude Code でできること

[Claude Code](#)はターミナル（黒い画面）で動作するAI開発ツールです。VSCodeのエディタ画面と組み合わせて使います。

ファイルを作る

「HTMLファイルを作って」と指示するだけで、必要なファイルを自動生成します。フォルダ構成まで考えてくれます。

コードを書く

「ボタンを押したらカウントが増える機能を追加して」のように、やりたいことを日本語で伝えればコードに変換します。

エラーを直す

画面が真っ白になった、ボタンが動かない。そんなときも「エラーを直して」と伝えれば、原因を特定して修正します。

コードを説明する

「このファイルは何をしているの？」と聞けば、コードの内容を日本語で解説してくれます。学習ツールとしても優秀です。

Security Tips — AI生成コードのレビュー責任

Claude Codeが書いたコードは、正しく動くとは限りません。セキュリティの穴が含まれていたり、意図しない外部通信を行ったりする可能性があります。AIが生成したコードであっても、最終的な品質判断は人間の責任です。「AIが書いたから大丈夫」ではなく、動作を確認し、疑問があれば質問する姿勢を持ってください。

従来の開発との決定的な違い

従来の開発では「プログラミング言語を学ぶ」が出発点でした。AI駆動開発では「何を作りたいか考える」が出発点です。技術的な実装はAIに任せ、人間は業務知識と判断力に集中できます。

参考リンク

[Claude Code 公式ドキュメント](#)[Visual Studio Code](#)

MINDSET

研修に臨む心構え

エラーは前進のサイン

赤い文字が表示されても慌てないでください。エラーが出るのは「何かを試した証拠」です。Claude Codeにエラーメッセージを見せれば、多くの場合そのまま解決してくれます。

完璧を目指さない

まず「動くもの」を作ることが最優先です。見た目は後から整えられます。機能も1つずつ足せばいい。最小限で動くもの（MVP）をまず手に入れてください。

隣の人と違って当然

同じ指示を出しても、AIの回答は毎回少し異なります。隣の人と見た目や動きが違っていても問題ありません。それがAI駆動開発のおもしろさでもあります。

わからなければ聞く

「こんなこと聞いていいのかな」という必要はありません。講師に声をかけてください。画面をそのままにしておいてもええと、原因がすぐに分かります。

COMPREHENSION CHECK

理解度チェック

Q1. AI駆動開発で、人間が担う最も重要な役割はどれですか

A プログラミング言語の文法を正確に書くこと

B 何を作りたいかを明確に伝え、生成物の品質を判断すること

正解です。AI駆動開発では、人間は「企画者」兼「レビュアー」です。要件を的確に伝え、出来上がったものが意図通りかを確認する力が求められます。

C AIが生成したコードを1行ずつ手動で修正すること

Q2. Claude Code はどこで動作しますか

A Webブラウザ上のチャット画面

B VSCodeのターミナル（黒い画面）

正解です。Claude CodeはVSCode下部のターミナル画面で動作します。エディタでファイルを確認しながら、ターミナルでAIと対話する形で開発を進めます。

C スマートフォンのアプリ

Q3. 開発中にエラーが表示された場合、最初にやるべきことは何ですか

A PCを再起動する

B 最初からやり直す

C Claude Code にエラーメッセージを見せて修正を依頼する

正解です。エラーメッセージにはAIが修正するための情報が含まれています。「このエラーを直して」と伝えるだけで、原因特定から修正まで行ってくれます。

[Top](#) [Session 2](#) [Session 3](#) [Session 4](#) [Session 5](#)

VSCode x Claude Code Hands-on Training Program

SESSION 02

環境構築ハンズオン

事前にインストールしたVSCodeを起動し、Node.jsとClaude Codeをセットアップします。ターミナルの最低限の操作もここで覚えます。

🕒 13:20 - 13:50 (30分) 📖 実践

ターミナルの基本操作と作業フォルダの作成

事前にインストールしたVSCodeを起動してください。[VSCode \(Visual Studio Code\)](#) の中にある[ターミナル](#)を使って、研修の作業フォルダを準備します。覚えるコマンドは3つだけです。

HANDS-ON

VSCodeでターミナルを開く

1. VSCodeを起動する
2. 上部メニューから「ターミナル」→「新しいターミナル」をクリック
3. 画面下部にターミナルが表示されます

VSCodeがまだ入っていない方へ

code.visualstudio.com からダウンロードしてインストールしてください。設定は初期値のまま進めて問題ありません。日本語化したい方は Ctrl+Shift+X で拡張機能を開き「Japanese」で検索してインストールしてください。

cd (移動する)

フォルダを移動するコマンドです。「change directory」の略。

```
cd Desktop
// デスクトップに移動
```

ls (一覧を見る)

今いるフォルダの中身を表示します。Windowsでは `dir` でも同じです。

```
ls
// フォルダの中身を表示
```

mkdir (フォルダを作る)

新しいフォルダを作ります。「make directory」の略。

```
mkdir training
// trainingフォルダを作成
```

Windowsの場合の補足

PowerShellでは `ls` も `dir` も使えます。`cd` と `mkdir` は Windows/Mac共通です。

EXERCISE

作業フォルダを作成する

1. ターミナルで `cd Desktop` と入力してEnter

2. `mkdir training` と入力してEnter
3. `cd training` と入力してEnter
4. `ls` と入力してEnter（まだ何も表示されないのが正常です）

達成条件

ターミナルの現在ディレクトリが `training` になっていれば成功です。パスの末尾に `Desktop/training` と表示されていることを確認してください。うまくいかない方は講師に声をかけてください。

Node.js をインストールする

Node.jsはJavaScriptというプログラミング言語をPC上で動かすための環境です。Claude Codeの動作に必要です。

HANDS-ON

Node.js のインストール

1. nodejs.org にアクセスする
2. 「LTS」と書かれた緑のボタンをクリック（推奨版です）
3. ダウンロードされたインストーラーをダブルクリック
4. インストーラーの指示に従って進める（すべて初期値でOK）
5. インストール完了後、VSCodeのターミナルを閉じて開き直す

インストールの確認

ターミナルで以下を実行してください。

```
node --version
// v22.x.x のようなバージョン番号が表示できれば成功です
```

達成条件

node --version を実行してバージョン番号（v22.x.x など）が表示されていれば成功です。

バージョンが表示されない場合

VSCodeを完全に閉じて再起動してください。それでもダメな場合は、インストール時にPATHの設定が反映されていない可能性があります。講師に声をかけてください。

Claude Code をセットアップする

いよいよClaude Codeをインストールし、研修用の組織アカウントでログインします。

インストール

ターミナルに以下のコマンドを貼り付けてEnterを押してください。

```
// Windows (PowerShell) の場合  
irm https://claude.ai/install.ps1 | iex
```

```
// Mac の場合  
curl -fsSL https://claude.ai/install.sh | sh
```

「Claude Code has been installed」と表示されたらインストール完了です。次のコマンドで確認してください。

```
claude --version  
// バージョン番号が表示されれば成功
```

つまずきポイント: コマンドが見つからない

「claude: command not found」と表示される場合は、VSCodeを完全に閉じて再起動してください。再起動後に再度 cd Desktop/training で作業フォルダに戻ることを忘れずに。

ログイン



claude と入力してEnter

Claude Codeが起動し、ログイン方法の選択画面が表示されます。



「Claude account with subscription」を選択

矢印キーで選択し、Enterで決定します。「Anthropic API key」は選ばないでください。



ブラウザで認証する

自動的にブラウザが開きます。研修事務局から届いた招待メールで作成したアカウントでログインしてください。



「Authorize」をクリック

組織の選択を求められた場合は「ClaudeCodeTutorial」を選択してください。



VSCodeに戻ってEnterを何度か押す

利用規約の確認を経て、入力待ち状態になれば完了です。

つまずきポイント: ブラウザが開かない

ターミナルに表示されるURLをコピーしてブラウザに貼り付けてください。c キーでURLがクリップボードにコピーされます。

Security Tips — 認証情報の取り扱い

ログインに使うアカウント情報やAPIキーは、パスワードと同じ機密情報です。画面共有中にターミナルへAPIキーを入力する場面では、共有を一時停止してください。スクリーンショットを撮る際も、キーやトークンが映り込んでいないか確認する習慣をつけてください。

動作確認

Claude Codeの入力欄に以下を入力してEnterを押してください。

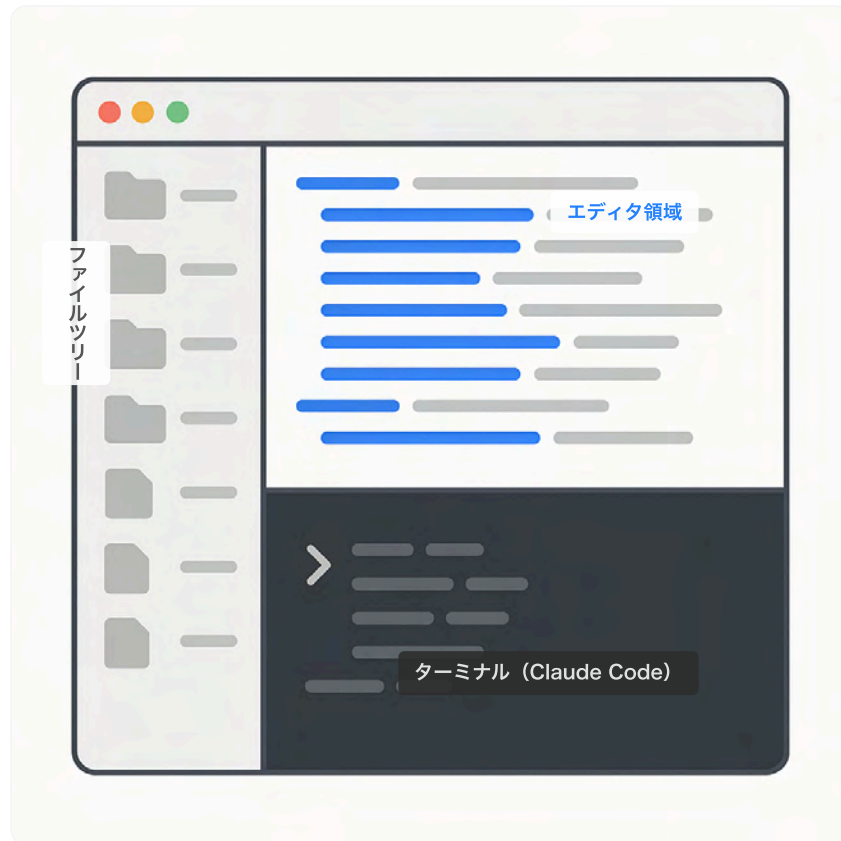
こんにちは。動作確認です。今日の日付を教えてください。

達成条件

Claude Codeに「こんにちは」と送り、日本語の応答が返ってくれば成功です。これで環境構築は完了しました。

REFERENCE

画面の見方と基本操作



VSCode画面の構成

- 上半分 = エディタ領域（ファイルの中身を表示）
- 下半分 = ターミナル領域（Claude Codeとの対話）
- 左サイドバー = ファイルツリー（フォルダ構成）

Claude Code の基本コマンド

- テキスト入力 + Enter = メッセージ送信
- Shift + Enter = 改行（送信しない）
- /clear = 会話をリセット
- /help = ヘルプを表示
- /exit = Claude Codeを終了
- Ctrl + C を2回 = 強制終了

許可を求められたら

Claude Codeがファイルを作成・編集する際に「Allow」や「Yes」と表示されることがあります。研修中は許可して進めてください。

TROUBLESHOOTING

困ったときは

claude コマンドが見つからない

VSCodeを完全に閉じて再起動してください。それでもダメな場合はPATHの設定が必要です。講師に声をかけてください。

ブラウザが自動で開かない

ターミナルに表示されるURLをコピーしてブラウザに貼り付けてください。c キーでURLがクリップボードにコピーされます。

認証がターミナルに反映されない

VSCodeのウィンドウに戻り、数秒待ってください。反映されない場合は Ctrl+C を押してから claude を再実行してください。

メッセージに応答しない

インターネット接続を確認してください。Ctrl+C を押してから claude を再実行してください。改善しない場合は講師に声をかけてください。

参考リンク

[Claude Code 公式ドキュメント](#)[VSCode ダウンロード](#)[Node.js 公式サイト](#)

COMPREHENSION CHECK

理解度チェック

Q1. ターミナルで「training」フォルダを作成するコマンドはどれですか

A cd training

B mkdir training

正解です。mkdir (make directory) で新しいフォルダを作成できます。

C ls training

Q2. Claude Code のログインで選択すべきオプションはどれですか

A Anthropic API key

B Claude account with subscription

正解です。研修用組織「ClaudeCodeTutorial」のサブスクリプションを利用してログインします。

C Skip login

Q3. Claude Code の会話をリセットするコマンドはどれですか

A /exit

B /clear

正解です。/clear で会話履歴がリセットされ、新しい状態からやり直せます。

C /help

[Top](#) [Session 1](#) [Session 3](#) [Session 4](#) [Session 5](#)

VSCode x Claude Code Hands-on Training Program

SESSION 03

ミニ業務アプリ開発

Claude Codeに日本語で要件を伝え、業務で実際に使えるアプリを作ります。Session 4のPDF比較ツールに向けた準備運動です。

🕒 14:00 - 14:50 (50分) 📄 実践

REFERENCE

AI駆動開発で作れるアプリの例

AppForgeには、AI駆動開発で作られた30種類の業務効率化アプリが掲載されています。PDF比較、CSV可視化、議事録整形、見積書生成、テキスト差分チェック —— どれもClaude Codeへの指示から生まれたものです。

このSessionではAppForgeに載っているようなミニアプリを、自分の手で1つ作ってみます。

 AppForge - AI駆動開発で構築した30本の業務効率化アプリ一覧

AppForge EXAMPLE

設備点検チェックリスト

点検項目の一覧表示、チェック、進捗率の可視化。紙の確認業務をデジタル化するアプリです。

AppForge EXAMPLE

テキスト差分チェッカー

2つのテキストの差分を色分け表示。仕様書の変更確認に使えます。Session 4のPDF比較と発想が同じです。

AppForge EXAMPLE

日報AIジェネレーター

箇条書きのメモからAIが日報を自動生成。報告書フォーマットに沿った文章をワンクリックで作成します。

AppForge EXAMPLE

パスワード強度チェック

パスワードの強度を視覚化し、安全なパスワードの生成も行えるセキュリティツールです。

PREPARATION / 5 MIN

作業フォルダを準備する

アプリごとにフォルダを分けて管理します。ターミナルで以下を実行してください。

```
// trainingフォルダの中に checklist フォルダを作成して移動する  
mkdir checklist  
cd checklist
```

VSCodeの「ファイル」メニューから「フォルダーを開く」で、今作った checklist フォルダを選択してください。

設備点検チェックリストを作る

業務に近いアプリとして、設備点検チェックリストを開発します。「誰が」「何のために」「どんな機能を」使うのかを整理してから、Claude Codeに渡すのがポイントです。

EXERCISE

Claude Code にアプリを作ってもらう

1. ターミナルで `claude` を実行してClaude Codeを起動する
2. 以下のプロンプトを入力してEnterを押す

ファイル作成の許可を求められたら

Claude Codeがファイルを作成・編集する際に「Allow（許可）」と表示されます。研修中は許可して進めてください。Y キーを押すか、「Yes」を選択すればOKです。

Shift + Enter で改行できます

長いプロンプトを入力するときは、Shift + Enter で改行しながら整形してください。Enter だけを押すとその時点で送信されます。プロンプトの構成を「背景 → 要件 → 制約」の順に書くと、意図が伝わりやすくなります。

// Claude Code に入力するプロンプト

設備点検チェックリストのウェブアプリを作ってください。

要件:

- 点検項目を一覧で表示する
- 各項目にチェックボックスをつける
- チェックした項目は打ち消し線で表示する
- 新しい項目を追加できる入力欄とボタンを設ける
- 画面上部に進捗率（完了数/全体数）を表示する
- デザインは白背景でシンプルに
- HTMLファイル1つで動くようにする

初期データとして以下の5項目を入れておいてください:

1. 電源ユニット動作確認
2. 冷却ファン回転チェック
3. 配線接続状態の目視確認
4. 温度センサー数値確認
5. 非常停止ボタン動作テスト

ブラウザで開いて動作確認する

Claude Codeがファイルを生成したら、左のファイルツリーに新しいHTMLファイルが表示されます。右クリック→「エクスプローラーで表示」→ダブルクリックでブラウザ表示してください。

達成条件 1

ブラウザに5つの点検項目が表示され、進捗率が「0/5（0%）」と表示されていれば成功です。

チェック操作を確認する

3つの項目にチェックを入れて、進捗率が「3/5（60%）」に変わることを確認してください。チェックした項目に打ち消し線が入り、項目を追加できれば、基本機能は動いています。

機能を追加する

以下のプロンプトを1つずつClaude Codeに入力してください。各指示の後にブラウザをリロード（F5）して動作を確認します。

// 追加指示1: 削除機能

各項目の右側に「削除」ボタンを追加してください。
削除する前に確認ダイアログを表示してください。

達成条件 2

各項目の右に「削除」ボタンが表示されていれば成功です。ボタンを押すと確認ダイアログが出て、OKを押すと項目が消えることを確認してください。

// 追加指示2: データ永続化

チェックリストのデータをブラウザのローカルストレージに保存してください。
ページをリロードしてもデータが消えないようにしてください。

達成条件 3

いくつかの項目にチェックを入れた状態でF5（リロード）してください。リロード後もチェック状態が保持されていれば成功です。

Security Tips — ブラウザ保存とデータの安全性

localStorageはブラウザ内にデータを保存する便利な仕組みですが、暗号化されていません。同じPCの別のユーザーや、ブラウザの開発者ツールから中身を簡単に閲覧できます。研修のチェックリストなら問題ありませんが、実業務で使う場合、顧客情報やパスワードなどの機密データをlocalStorageに保存するのは避けてください。機密データを扱うアプリでは、サーバー側での暗号化保存を検討する必要があります。

うまくいかなかったときは

「さっきの変更を戻して、もう一度やり直してください」と伝えればOKです。Claude Codeは会話のコンテキスト（文脈）を覚えていてくれます。

覚えておくと効く Claude Code の使い方

TECHNIQUE 01

エラーはそのまま貼り付ける

ブラウザやターミナルにエラーが出たら、メッセージをそのままコピーして Claude Code に渡してください。「エラーが出ました」だけでは原因を特定できません。エラー文を見せれば Claude Code が修正コードを出してくれます。

TECHNIQUE 02

/clear で仕切り直す

やり取りが長くなって混乱してきたら /clear と入力してください。会話履歴がリセットされ、まっさらな状態から指示を出せます。Claude Code の応答が的外れになり始めたときに有効です。

TECHNIQUE 03

Esc キーで生成を中断

Claude Code が意図と違うコードを書き始めたら、Esc キーで途中停止できます。止めてから「そうではなく、こうしてください」と軌道修正するほうが、完了まで待つより効率的です。

TECHNIQUE 04

ファイルツリーの変化を見る

Claude Code がファイルを作成・編集すると、VSCode の左サイドバーにあるファイルツリーにリアルタイムで反映されます。どんなファイルが生まれたか、どのファイルが変わったかを確認する習慣をつけてください。

Claude Code は直近の会話を「コンテキスト」として保持しています。最初に伝えた要件や、途中で出した追加指示を踏まえて次の回答を生成するため、「さっきの」「先ほどの」といった指示代名詞が通じます。

ただし、コンテキストには上限があります。やり取りが何十往復にもなると Claude Code が古い指示を忘れ始め、意図と違う修正をすることがあります。そのタイミングが /clear の出番です。リセット後は必要な前提を改めて伝え直してください。

プロンプトの書き方 ― 基本3原則



PRINCIPLE 01

具体的に書く

「いい感じにして」より「背景を白、文字を黒、ボタンを青にして」のほうが意図通りの結果になります。色、サイズ、位置を明示してください。

PRINCIPLE 02

一度にひとつずつ

機能追加は1つずつ。一度に5個頼むと、どれかがうまくいかなかったときに原因が分かりにくくなります。

PRINCIPLE 03

結果を確認する

指示を出したら必ずブラウザをリロード（F5キー）して動作を確認してください。確認なしに次の指示を出すと、問題が積み重なります。

Claude Code コマンド早見表

ハンズオン中に使う頻度が高い操作をまとめます。

Enter で送信

Shift + Enter で改行

メッセージを確定して送信します。長いプロンプトを途中で送ってしまわないよう注意してください。

送信せずに改行します。要件を箇条書きで整理するときに使います。

/clear で会話リセット

コンテキストを初期化します。やり取りが混乱したときや、新しいタスクに切り替えるときに使います。

Esc で生成中断

Claude Codeがコードを書いている途中で止められます。方向性が違うと気づいたら即座に中断してください。

次のSession 4で本格アプリに挑戦します

ここで覚えた「要件整理→指示→確認→改善」のサイクルを、次のPDF文書比較ツールでも使います。HTMLファイル1つのアプリから、サーバーを使った本格アプリへステップアップします。

CHALLENGE / 10 MIN（余裕のある方向け）

もう1つアプリを作ってみる

時間が余った方は、新しいフォルダを作って別のアプリに挑戦してみてください。

- ・「簡単なメモアプリを作って。メモの追加・削除ができて、ローカルストレージに保存するもの」
- ・「パスワードの強度をチェックするツールを作って。入力したパスワードの長さ・文字種で強弱を表示」
- ・「2つのテキストエリアに文章を入力すると、差分を色分け表示するツール」

参考リンク

AppForge（業務アプリ集）

Claude Code 公式ドキュメント

COMPREHENSION CHECK

理解度チェック

Q1. Claude Code にアプリを作ってもらう前に、最初にやるべきことは

A プログラミング言語を決める

B 何を表示し、どんな操作ができるかを日本語で整理する

正解です。「誰が」「何のために」「どんな機能を」使うのかを整理してからClaude Codeに伝えると、意図に近いアプリが一度で出てきやすくなります。

C デザインを詳細に決める

Q2. Claude Code への指示で望ましいのはどれですか

A 「いい感じにして」

B 「各項目の右側に削除ボタンを追加してください」

正解です。具体的で、1つの機能に絞った指示です。結果を確認してから次の指示に進めます。

C 「ボタン追加、色変更、アニメーション、フォント変更を同時にやって」

[Top](#) [Session 1](#) [Session 2](#) [Session 4](#) [Session 5](#)

VSCode x Claude Code Hands-on Training Program

SESSION 04

PDF文書比較ツールを作る

講師が作ったPDF比較ツールのデモを見た後、同じアプリをゼロから自分のプロンプトで開発します。Claude Code への「指示の出し方」が試される実践セッションです。

🕒 14:50 - 16:10 (80分) 📄 実践

OVERVIEW

このセッションの進め方

Session 3ではHTMLファイル1つで動くアプリを作りました。ここからは、サーバーとAI APIを使った本格的なウェブアプリに挑戦します。

ただし、今回は完成品のコードを渡して改修する形ではありません。講師のデモを見てから、自分自身のプロンプトでゼロからアプリを開発していただきます。

講師デモを見る

完成版のPDF比較ツールを講師が動かします。画面構成と操作の流れを観察してください。

ダミーデータを確認する

比較対象となるPDFの3セットを全員で開き、中身と差分のポイントを把握します。

自分のプロンプトで開発する

デモで見た画面をゴールに、Claude Codeへプロンプトを書いてアプリを構築します。

完成したアプリを共有する

動作確認ができればスクリーンショットを撮り、ZOOMチャットで全員に共有します。

完成品は参考用として配布済みです

講師が作った pdf-compare-app フォルダは参照用として配布しています。行き詰まったときにコードを見る用途に使ってください。メインの学びは「自分のプロンプトで作ること」にあります。

講師デモを見る

講師がPDF比較ツールの完成版を操作します。画面を見ながら、以下の点を意識して観察してください。

観察ポイント 01

画面構成

PDFをアップロードするエリアが2つ、比較実行ボタン、結果の表示エリアがどう配置されているかを見てください。

観察ポイント 02

操作の流れ

PDFを選ぶ → 比較を実行 → 結果を読む → レポートやメールに変換、という一連の流れを追ってください。

観察ポイント 03

比較結果の構造

概要、一致点、相違点（重要度つき）、片方にだけある記述、の4セクションに分かれて表示されます。

観察ポイント 04

変換機能

比較結果をビジネスレポートやメール文面に変換するボタンがあります。上司への報告を自動化する発想です。

ACTION

デモ画面のスクリーンショットを撮っておく

後の開発で「この画面を参考にしてください」とClaude Codeに見せる材料になります。Windows の場合は Win + Shift + S、Mac の場合は Cmd + Shift + 4 で画面の一部をキャプチャできます。

ダミーデータの確認と開発準備

3セットのサンプルPDF

配布フォルダの `sample-data/` に3種類の文書ペアが入っています。講師と一緒にそれぞれの内容を開いて確認しましょう。比較対象の文書にどんな差分が含まれているかを事前に把握しておく、完成後の動作検証がスムーズになります。

セット	文書の種類	ファイル名	注目すべき差分
set1	設備点検手順書	set1_設備点検手順書_v1.0.pdf set1_設備点検手順書_v2.0.pdf	v2.0でGMP対応の追加、重量検査装置の手順追加、充填精度基準の変更
set2	業務委託契約書	set2_業務委託契約書_A社案.pdf set2_業務委託契約書_B社案.pdf	報酬条件、責任範囲、契約期間の条項の差異
set3	技術仕様書	set3_技術仕様書_HX-200現行.pdf set3_技術仕様書_HX-300次期.pdf	型番の違い、スペック数値、新機能の有無

開発フォルダの準備

自分専用の開発フォルダを作り、必要なファイルだけ配置します。



新しいフォルダを作る

デスクトップなど好きな場所に `my-pdf-compare` フォルダを作成してください。



sample-data をコピーする

配布フォルダ内の `sample-data/` を丸ごと `my-pdf-compare/` にコピーしてください。



.env ファイルを作成する

VSCodeでフォルダを開き、ルートに `.env` ファイルを作って講師配布のAPIキーを記入します。



Claude Code を起動する

VSCodeのターミナルで `claude` と入力してClaude Codeを起動してください。

```
// .env ファイルの内容（講師配布のキーを貼り付け）
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxx
```

APIキーは外部に公開しないでください

.env ファイルに記載するAPIキーは認証情報です。メールやチャットに貼り付けたり、GitHubにアップロードしたりしないでください。研修終了後にキーは無効化されます。

Security Tips — APIに送信されるデータを意識する

このPDF比較ツールは、アップロードされたPDFの全文をClaude APIに送信して分析しています。研修ではダミーデータを使うため問題ありませんが、実業務で使う場合は注意が必要です。社外秘の文書、個人情報を含む書類、法的に外部送信が制限されるデータは、APIに送信してよいかセキュリティポリシーを事前に確認してください。.envファイルをGitにコミットしないよう.gitignore に追加する運用も習慣にしてください。

自分のプロンプトで開発する

ここが本セッションの核です。デモで見た画面をゴールとして、Claude Codeに「何を作りたいか」を自分の言葉で伝えてください。

プロンプトをいきなり完璧に書く必要はありません。まず大まかな指示を出し、足りない部分を追加で伝える。Session 3で身につけた「具体的に / 一度にひとつ / 結果を確認」のリズムがここでも使えます。

プロンプトに含めたい7つの要素

以下の要素をヒントにして、自分なりのプロンプトを組み立ててください。一度に全部伝えても、数回に分けて伝えても構いません。

01

役割

Claude Codeに「どんな開発者として振る舞ってほしいか」を伝えます。

例: あなたはNode.jsのウェブアプリ開発者です

02

目的

作りたいものを端的に伝えます。一文で言い切れるのが理想です。

例: 2つのPDFを比較分析するウェブアプリを作ってください

03

条件

使う技術やポート番号など、守ってほしい制約を伝えます。

例: Express で localhost:3001 に立てて、Anthropic SDK を使用

04

文脈

なぜ作るのか、どんな場面で使うのかを補足します。スクリーンショットの説明も有効です。

例: 先ほどのデモのスクリーンショットを参考にしてください

05

ダミーデータ

テスト用のデータがどこにあるかを伝えます。

例: sample-data/ に3セットのテスト用PDFがあります

06

出力形式と場所

ファイル構成の指定です。サーバー側とフロント側を分ける構成も伝えられます。

例: server.js をルートに、フロントは public/ に配置

07

APIの使用方法

外部APIをどう使うかを伝えます。キーの場所も知らせてください。

例: `.env` の `ANTHROPIC_API_KEY` で *Claude API* を呼び出し、*PDF*を分析

以下は全要素を盛り込んだ場合の一例です。丸写しではなく、自分の言葉で書き換えて使ってください。

// 参考プロンプト（あくまで一例）

あなたはNode.jsのウェブアプリ開発者です。
以下の要件でPDF文書比較ツールを作ってください。

【目的】

2つのPDFをアップロードし、AIが内容を比較分析するウェブアプリ

【技術条件】

- Express でサーバーを構築、localhost:3001 で起動
- Anthropic SDK で Claude API を呼び出す
- multer でPDFアップロードを処理
- pdf-parse でPDFテキストを抽出

【ファイル構成】

- server.js (サーバー)
- public/index.html (画面)
- public/style.css (デザイン)
- public/app.js (操作と表示)
- package.json

【機能】

- PDFを2つアップロードするUI
- 比較実行ボタン
- 結果表示（概要、一致点、相違点、固有の記述）
- 相違点に重要度（高/中/低）を付与
- レポート形式への変換ボタン
- メール形式への変換ボタン

【データ】

- sample-data/ にテスト用PDFが3セットあります
- APIキーは `.env` の `ANTHROPIC_API_KEY` に設定済みです

開発の進め方

プロンプトを送信すると、Claude Codeがファイルを生成し、必要なパッケージのインストールまで実行してくれます。生成が終わったら、ターミナルで `npm start` を実行してブラウザで `http://localhost:3001` を開いてください。画面が表示されれば第一段階は成功です。

動いたらset1で比較を試す

sample-data/ の set1（設備点検手順書 v1.0 と v2.0）を使って比較を実行してください。結果が表示されればアプリは完成です。

もし結果の表示がおかしかったり、機能が足りなかったりしたら、追加のプロンプトで修正を依頼してください。一度で完璧にならなくて当然です。

server.js を変更したらサーバーを再起動

Claude Codeがserver.jsを修正した場合は、ターミナルで `Ctrl+C` → `npm start` でサーバーを再起動してください。public/ フォルダだけの変更ならブラウザのリロード（F5）で反映されます。

TIPS

プロンプトの中のプロンプト

このアプリの server.js には、Claude APIに送る分析指示（プロンプト）が含まれています。「AIに指示を出す文章を、別のAIに書いてもらう」構造です。ここを調整すると分析の切り口が変わります。「法務観点で分析して」「安全管理の視点で比較して」といった指示も有効です。

TIPS

完成品を参照する

思うように動かない場合は、配布済みの pdf-compare-app フォルダのコードを見て構いません。server.js のプロンプト構造や public/ のUI構成を参考にして、自分のアプリに反映してください。

参考リンク

[AppForge（業務アプリ集）](#)[Claude Code 公式ドキュメント](#)[Claude API リファレンス](#)

完成と共有

エラーが出た場合

アプリの起動や比較実行でエラーが出るのは珍しくありません。ブラウザやターミナルに表示されるエラーメッセージをそのまま Claude Codeに貼り付けてください。

```
// エラー対応のプロンプト例
```

```
npm start したら以下のエラーが出ました。修正してください。
```

```
Error: Cannot find module 'express'  
...
```

自分でエラーの意味を理解する必要はありません

エラーメッセージの全文を Claude Codeに渡すだけで、原因の特定と修正を一括で行ってくれます。「何が起きているか」よりも「エラーを正確にコピペすること」が大事です。

全員の動作確認

set1のPDFで比較結果が表示されたら完成です。講師が全員の画面を順番に確認します。

FINAL ACTION

ZOOMチャットにスクリーンショットを共有

完成したアプリの比較結果画面をスクリーンショットで撮影し、ZOOMのチャットに貼り付けてください。全員のアプリがどんな見た目になったか、比較するのも面白い発見になります。同じ要件でもプロンプトの書き方次第でUIや分析結果の見せ方が変わることを体感してください。

余裕がある方は

set2（業務委託契約書）やset3（技術仕様書）でも比較を試してみてください。文書の種類が変わるとAIの分析観点が変わることが体感できます。独自の機能追加（CSV出力、比較履歴の保存など）にも挑戦してみてください。

COMPREHENSION CHECK

理解度チェック

Q1. Claude Codeでアプリをゼロから開発するとき、プロンプトに含めると効果が高い要素はどれですか

A プログラミング言語の文法を詳しく解説する

B 目的、技術条件、ファイル構成を具体的に伝える

正解です。「何を作りたいか（目的）」「どんな技術を使うか（条件）」「どんなファイル構成にするか（出力形式）」を具体的に指定すると、意図に近いアプリが生成されます。今回学んだ7要素がその基本です。

C 「いい感じに作って」と一言だけ伝える

Q2. アプリ開発中にエラーが出た場合、最も効率的な対処法はどれですか

A エラーメッセージ全文をClaude Codeに貼り付けて修正を依頼する

正解です。エラーメッセージにはClaude Codeが原因を特定するための情報が含まれています。全文を渡すことで、修正コードまで一括で提案してくれます。

B コードを全部消して最初からやり直す

C エラーメッセージを自分で翻訳してからコードを手動で直す

Q3. Session 3のチェックリストとPDF比較ツールの構成上の違いはどれですか

A 使っているプログラミング言語が異なる

B PDF比較ツールはサーバー（Express）と外部API（Claude）を使っている

正解です。チェックリストはHTMLファイル1つで完結していましたが、PDF比較ツールはサーバーがPDFの処理とAPI通信を担当し、フロントエンドが画面表示を担当する構成です。

C PDF比較ツールのほうがファイル数が少ない

SESSION 05

生成AI資格ガイド

生成AIを業務で活用していく上で、リテラシーを体系的に証明できる資格をご紹介します。まずは「生成AIパスポート」の取得がおすすめです。その先のキャリア方向に合った資格も確認しましょう。

 16:20 - 17:00 (40分)  座学

まずは「生成AIパスポート」から

本日の研修で体験した通り、生成AIは「使い方を知っている人」と「知らない人」の間で成果に大きな差が出ます。その「使い方を知っている」ことを客観的に証明する手段が資格です。

生成AI関連の資格は数多くありますが、最初の一步としておすすめしたいのが「生成AIパスポート」です。

生成AIパスポートとは

一般社団法人生成AI活用普及協会（GUGA）が実施する、生成AIに関するリテラシーを問う検定試験です。AIを日常業務に活かすための最初の一步として位置づけられています。

項目	内容
出題範囲	AIの基礎知識、生成AIの仕組み、プロンプト設計の基本、倫理・法的リスク、ビジネス活用
試験形式	オンライン / 多肢選択式 / 60問 / 60分
難易度	初級（IT未経験者でも合格可能）
受験費用	11,000円（税込）
合格基準	正答率70%以上

本日の研修内容が直結します

今日体験した「プロンプト設計」「AI駆動開発」「APIの仕組み」は、生成AIパスポートの出題範囲と重なる部分が多くあります。研修の知識が新鮮なうちに受験していただくと、効率よく合格を目指せます。資格の取得を強制するものではありませんが、ぜひご検討ください。

生成AIパスポートを取得済みの方、あるいは取得後に「次は何を目指すか」を考えたい方のために、7つの方向性と15の資格を俯瞰してご紹介します。

7つの方向性と15の資格

A

AI活用リテラシー —— AIの全体像を広く押さえたい人向け

生成AIパスポートと最も地続きの領域です。G検定は「AIの全体像を体系的に語れるようになる」資格で、ビジネス職や管理職の方に向いています。

#	資格名	主催	概要	難易度	費用
1	G検定	JDLA	ディープラーニングを事業に活かす知識。累計受験者13万人超	中級	13,200円
2	全統AI	全統AI実行委	生成AIのビジネス活用力を測る全国統一試験	初~中	無料~低額
3	AI検定	サーティファイ	AI基礎知識~ビジネス活用。50分30問リモートWebテスト	初級	4,400円
4	ITパスポート	IPA	IT全般の国家試験。2021年からAI分野を大幅強化	初級	7,500円

B

プロンプト・実務活用 —— 日々のAI活用スキルを証明したい人向け

生成AIパスポートの知識を「実務で使える形」に昇華する方向です。本日の研修で体験したプロンプト設計のスキルを、資格として証明できます。

#	資格名	主催	概要	難易度	費用
5	PEP検定	プロンプトエンジニア協会	プロンプト設計の体系的知識と実践スキルを認定	中級	11,000円
6	JDLA Generative AI Test	JDLA	生成AIの理解度を測る20分ミニテスト。G検定への足がかり	初級	2,200円

C

データサイエンス —— データ分析の素養を固めたい人向け

AIが出す数値の妥当性をご自身で判断できるようになる方向です。統計検定2級はAI関連資格の中でも数理的な強度が頭一つ抜けており、データを扱う部門の方には特に有効です。

#	資格名	主催	概要	難易度	費用
7	DS検定	データサイエンティスト協会	ビジネス力・DS力・DE力の3軸。リテラシーレベルだが骨太	中級	10,000円
8	統計検定2級	統計質保証推進協会	大学基礎レベルの統計学。仮説検定・回帰分析・確率分布	中級	7,000円

AI開発エンジニアリング —— AIを「作る側」に回りたい人向け

エンジニア職やR&D部門の方が対象です。E資格は認定プログラム修了が必須のため学習コストは高くなりますが、AI開発人材としての市場価値は大きく上がります。

#	資格名	主催	概要	難易度	費用
9	E資格	JDLA	DL理論の理解と実装能力を認定。認定プログラム修了が受験要件	上級	33,000円
10	AI実装検定 A級	AI実装検定実行委	NN基礎構造の理解とPython実装力。E資格の手前のステップ	中級	14,850円

クラウドAI —— 自社のAI基盤構築に関わる人向け

自社でAWS/Azureを使っている場合、そのまま業務に直結します。どちらもFundamentalsレベルですので、開発経験がなくても合格圏に入ることができます。グローバルで通用する認定です。

#	資格名	主催	概要	難易度	費用
11	AWS Certified AI Practitioner	AWS	2024年新設。AI・ML・生成AIの基本とAWSサービス活用	初~中	16,500円
12	Azure AI Fundamentals (AI-900)	Microsoft	Azure上のAI・MLサービス基礎。無料バウチャーの機会あり	初級	13,200円

セキュリティ・ガバナンス —— AI活用の「守り」を固めたい人向け

生成AIを業務で使う以上、情報漏えいや不正利用のリスクは避けて通れません。AI活用を推進する立場の方が「守り」の基盤を持つ意味は大きいと考えます。

Security Tips —— AI活用における「守り」の優先度

本日の研修で体験した通り、AIで便利なツールを短時間で作れるようになりました。一方で、作ったツールが社外秘データを外部APIに送信していたり、アクセス制御が甘い状態で社内展開されたりすれば、情報漏えいの原因になります。AI活用の「攻め」と「守り」は両輪です。ツールを作る側にセキュリティの素養があれば、IT部門との連携もスムーズに進みます。

#	資格名	主催	概要	難易度	費用
13	情報セキュリティマネジメント試験	IPA	ユーザー企業の情報管理担当者向け国家試験。CBT通年受験可	中級	7,500円

プログラミング・開発 —— AI時代の開発スキルを武器にしたい人向け

本日の研修ではClaude Codeを使ってアプリを開発しました。コードを読み書きするスキルを正式に証明したい方向けの資格です。GitHub Copilot認定は2024年に新設された資格で、AIを開発ワークフローに組み込むスキルの証明として注目されています。

#	資格名	主催	概要	難易度	費用
14	Python 3 エンジニア認定試験	Pythonエンジニア育成推進協会	Python文法・ライブラリ知識を3段階で認定	初~中	各11,000円
15	GitHub認定資格	GitHub, Inc.	Foundations / Actions / Security / Admin / Copilotの5種	初~中	各\$99

PROPOSAL / 5 MIN

「次の一手」3パターン

キャリア志向に合わせて、生成AIパスポートの次に目指す資格を3パターンでご案内します。

PATTERN 01

横に広げる

まず G検定 or ITパスポートで
周辺知識を固める

次に 情報セキュリティマネジメント試験でガバナンス
面をカバー

非エンジニア・管理職の方におすすめ
です。「AIを推進する側」としての信
頼性を組織内で確立できます。

PATTERN 02

縦に深める

まず DS検定 or 統計検定2級
で数理基盤を作る

次に G検定 → E資格と段階的
に進む

技術職・データ分析職の方におすすめ
です。「使う」から「評価・設計でき
る」レベルへステップアップできま
す。

PATTERN 03

実務に直結

まず PEP検定 or Generative
AI Testで生成AI活用力を
証明

次に AWS AI Practitioner or
Azure AI-900で自社クラ
ウド環境のAI活用を学ぶ

全職種の方におすすめです。「今の仕
事」で使えるスキルを最短距離で証明
できます。

まずは生成AIパスポートから

どのパターンを選ぶにしても、生成AIパスポートが出发点です。本日の研修内容と出題範囲が重なっているため、記憶が新しいうちの受験が効率的です。各資格の公式サイトURLは次のセクションにまとめています。

REFERENCE

資格公式サイト一覧

#	資格名	公式サイト
0	生成AIパスポート	guga.or.jp/outline
1	G検定	jdla.org/certificate/general
2	全統AI	zentou.ai
3	AI検定（サーティファイ）	sikaku.gr.jp/ai
4	ITパスポート	jitec.ipa.go.jp
5	PEP検定	prompt.or.jp/pep
6	JDLA Generative AI Test	jdla.org/certificate/generativeai
7	DS検定	datascientist.or.jp
8	統計検定2級	toukei-kentei.jp
9	E資格	jdla.org/certificate/engineer
10	AI実装検定 A級	kentei.ai
11	AWS Certified AI Practitioner	aws.amazon.com/certification
12	Azure AI Fundamentals	learn.microsoft.com
13	情報セキュリティマネジメント試験	ipa.go.jp/shiken
14	Python 3 エンジニア認定試験	odyssey-com.co.jp
15	GitHub認定資格	resources.github.com

TIPS

作成したアプリを社内でも共有・公開するには

ご注意

ここで紹介する内容は、あくまで一般的な情報です。そのまま手順通りに実施すれば社内公開できる、という趣旨ではありません。実際の導入にあたっては、社内のIT部門やセキュリティポリシーとの調整が必要になります。「こういう選択肢がある」という見取り図としてお読みください。

研修で作成したアプリは、現時点では自分のPCだけで動いています。これを同僚にも使ってもらうには、大きく3つの段階があります。

STEP 1

ローカル実行のまま見せる

方法 自分のPCでサーバーを起動し、画面共有やスクリーンショットで共有する

利点 追加の準備が不要。今日の研修内だけでも実現できる

まず動いているものを見せたい、フィードバックをもらいたいという段階に適しています。

STEP 2

社内ネットワークで共有する

方法 社内サーバーやイントラネット上にアプリを配置する

例 社内のWindows ServerにNode.jsをインストールして常時起動する、Docker化して社内環境にデプロイする、など

チーム内で日常的に使いたい段階。IT部門との連携が必要になります。

STEP 3

クラウドで公開する

方法 AWS、Azure、Vercel等のクラウドサービスにデプロイする

検討事項 認証・アクセス制限、APIキー管理、利用コスト、データの保管場所に関するポリシー確認

部門横断や外部パートナーとの共有を想定する場合。セキュリティ審査を含む正式なプロセスを経る形が一般的です。

共有時に気をつけたいこと

- APIキー（Anthropic APIキーなど）をコードに直接書かない。環境変数や社内のシークレット管理ツールを利用する
- 社外秘の文書や個人情報を扱う場合、外部APIへの送信内容がセキュリティポリシーに適合するか確認する
- 利用者が増えるとAPI呼び出し量とコストが増加する。利用量の上限設定や課金体制を事前に整理しておく
- 「誰がアクセスできるか」の範囲を明確にする。社内限定であっても認証の仕組みは入れておくのが望ましい

まずはSTEP 1から

今日の研修で作ったアプリは、まずは画面共有で同僚に見せるところから始めてみてください。「AI駆動開発でこんなものが作れる」と伝えるだけでも、チーム内のAI活用への関心を高めるきっかけになります。本格的な社内展開を検討する際は、IT部門と相談しながら進めていただければと思います。

COMPREHENSION CHECK

理解度チェック

Q1. 非エンジニアが生成AIパスポートの次に目指すべき推奨パターンは

A E資格 → AI実装検定A級

B G検定 or ITパスポート → 情報セキュリティマネジメント試験

正解です。パターン01「横に広げる」に該当します。G検定やITパスポートで周辺知識を固め、情報セキュリティマネジメント試験でガバナンス面をカバーする流れが、AI推進者としての信頼性確立に直結します。

C Python 3 エンジニア認定試験 → GitHub認定資格

D 統計検定2級 → DS検定

Q2. G検定の説明として正しいものはどれか

A AIの実装能力を認定するプログラミング試験

B ディープラーニングを事業に活かす知識を問うJDLAの試験

正解です。JDLAが主催するG検定は、AIの全体像を体系的に語れるようになる資格です。累計受験者13万人超で、ビジネス職や管理職に向いています。

C Google Cloud のAIサービスに特化した認定試験

D 政府が実施するAI国家試験

Q3. 2024年に新設され、生成AIを開発ワークフローに組み込むスキル証明として注目されている資格は

A PEP検定

B AWS Certified AI Practitioner

C GitHub Copilot認定資格

正解です。GitHub認定資格の1つとして2024年に新設されました。AIペアプログラミングツールのCopilotを開発ワークフローに組み込むスキルを証明する資格として注目度が高まっています。

D JDLA Generative AI Test

[Top](#) [Session 1](#) [Session 2](#) [Session 3](#) [Session 4](#)

VSCode x Claude Code Hands-on Training Program

SESSION 06 / DAY 2

自由開発 & 成果発表

Session 5で企画したアプリを、Claude Codeを使って実際に開発します。完成度よりも「動くもの」を手に入れることが目標です。最後に全員で成果を共有します。

🕒 13:00 - 16:00 (休憩含む) 📄 実践 + ワーク

開発の進め方

MVP (Minimum Viable Product) で始める

最初から完成形を目指す必要はありません。まず最小限の機能だけ動かし、そこから1つずつ積み上げてください。「機能を1つ足して、確認する」のサイクルを繰り返すのが最も確実な進め方です。



Security Tips — 開発中に気をつけること

自由開発ではAPIキーを扱う場面が出てきます。3つのポイントを守ってください。(1) APIキーはコードに直接書かず、.env ファイルで管理する。(2) スクリーンショットや画面共有でAPIキーが映り込まないように注意する。(3) デモ用のサンプルデータに本物の個人情報や社外秘情報を使わない。発表時のスクリーンショットにも同じ配慮が必要です。

13:00 - 13:10 — プロンプトの最終調整

Session 5で作ったプロンプトを見直し、必要があれば修正してください。

13:10 - 13:30 — 最小版を生成

プロンプトを投入してアプリの骨格を作ります。ブラウザで動くことを確認してください。

13:30 - 14:30 — 機能追加

1つずつ機能を足していきます。講師が巡回していますので、いつでも相談してください。

14:30 — 中間チェック

隣の人に画面を見せて、使い勝手の意見をもらいます。5分程度。

14:30 - 15:00 — 仕上げ

見た目を整えたり、発表で見せるデモデータを入れたりして仕上げてください。

TROUBLESHOOTING TIPS

開発中に困ったら

Claude Codeが止まったように見える

長いコードを生成中の場合、数十秒かかることがあります。ターミナルにテキストが少しずつ表示されていれば正常です。完全に止まっている場合は Ctrl+C で中断し、もう一度指示を出してください。

意図と違うものが生成された

「さっきの変更を戻してください」と伝えた上で、指示を具体的に直してください。「ボタンの下に表を追加」のように位置を明示すると改善します。

エラーが出て動かない

エラーメッセージをそのままClaude Codeに伝えてください。「ブラウザのコンソールにエラーが出ています。直してください」でも通じます。

会話が長くなりすぎた

`/clear` で会話をリセットし、現在のファイルの状態から仕切り直せます。「このファイルに〇〇の機能を追加して」のように、改めて指示を出してください。

成果発表

1人5～7分で発表します。以下の構成で話してください。

PRESENTATION FORMAT

発表の流れ（1人5～7分）

1. アプリの目的: 「何を解決するアプリか」を一言で
2. デモ: 実際にブラウザでアプリを操作して見せる
3. 工夫した点: 特にうまくいったプロンプトや工夫を共有
4. 苦労した点: うまくいかなかったこと、どう対処したか
5. 今後の改善案: 「次にやるならこの機能を追加したい」

聞く側のポイント

「自分の業務にも使えそうか」「この機能があるとさらに便利では」という視点でフィードバックを伝えてみてください。発表者にとって、実務観点の意見が一番の気づきになります。

SESSION 07 / DAY 2

アプリを届けるには

作ったアプリを自分のPCの外に出し、チームや社内で使えるようにするまでの道筋を理解します。「完成した後どうするか」に答えるセッションです。

🕒 16:00 - 17:00 (60分) 📖 座学

OVERVIEW

公開までの3つの段階

研修で作ったアプリは、今はまだ自分のPC（ローカル）でしか動きません。他の人にも使ってもらうには、アプリをサーバーに置く必要があります。段階的に考えてみましょう。



STEP 1

ローカル実行

今日やったこと。自分のPCでHTMLファイルをブラウザで開く方法です。自分しか使えません。

- サーバー不要
- すぐに動く
- 他の人には共有できない

STEP 2

社内サーバー

社内ネットワーク上のサーバーにアプリを置き、同じネットワーク内の人がブラウザでアクセスできるようにします。

- 社内の人が見える
- 社外からはアクセス不可
- IT部門との連携が必要

STEP 3

クラウド (AWS)

Amazon Web Servicesなどのクラウドサービスにアプリを配置します。インターネット経由でどこからでもアクセス可能になります。

- どこからでもアクセス可能
- スケーラビリティが高い
- セキュリティ設定が必要

デプロイの実際

HTMLだけのアプリの場合

研修で作ったようなHTML 1ファイルのアプリは、最もシンプルにデプロイできます。ファイルをサーバーに置くだけで動きます。

- 社内のファイルサーバーにHTMLを置き、URLを共有する
- GitHub Pagesのような静的サイトホスティングを利用する
- AWS S3 + CloudFront で公開する

サーバー処理が必要なアプリの場合

データベースに保存する、AI APIを呼び出すなど、サーバー側の処理が必要なアプリは、サーバーを立てる必要があります。

- Python (Flask/Django) やNode.js (Express) などのバックエンドフレームワーク
- AWS EC2、ECS、Lambda などのコンピューティングサービス
- データベース (RDS、DynamoDB など)

すべてを自分でやる必要はありません

AI駆動開発で「何を作るか」を考え、動くプロトタイプを作るところまでが皆さんの役割です。インフラ構築やセキュリティ設定は、IT部門や外部パートナーと連携して進めるのが現実的です。

LECTURE

Terraform —— インフラをコードで管理する

Terraformは、サーバーやネットワークの構成をコード（設定ファイル）で定義し、コマンド一発で自動構築するツールです。

従来のインフラ構築

AWSの管理画面にログインし、マウスでポチポチ設定していく。手順書が必要で、再現性が低い。

Terraformによるインフラ構築

設定ファイルにインフラの構成を記述し、`terraform apply`と打つだけ。同じ構成を何度でも再現できます。

以下は AWS に EC2 サーバーを1台立てる設定の概要です。Claude Code でこうした設定ファイルも生成できます。

```
resource "aws_instance" "app_server" {
  ami          = "ami-0abcdef1234567890"
  instance_type = "t3.micro"

  tags = {
    Name = "my-web-app"
  }
}
```

この数行で、AWSにサーバーが1台立ちます。`terraform apply`を実行するだけです。削除するときは `terraform destroy` でサーバーごと消えます。

AI駆動開発 × Terraform

Claude Code はTerraformの設定ファイルも生成できます。「AWSにFlaskアプリをデプロイする構成を作って」と指示すれば、EC2、セキュリティグループ、ロードバランサーの設定をまとめて出力します。AI駆動開発はアプリ開発にとどまらず、インフラ構築にも適用できる領域です。

Security Tips —— インフラ設定にも機密管理が必要

Terraformの設定ファイルにAPIキーやデータベースのパスワードを直接書くのは厳禁です。AWS Secrets ManagerやParameter Storeを使い、コードと機密情報を分離するのが前提になります。インフラ設定ファイルもアプリのコードと同様に、セキュリティレビューの対象に含めてください。

社内公開までの現実的なステップ

プロトタイプが動いた後、社内で実際に使えるようにするまでの流れを整理します。

作る

レビュー

テスト

社内公開

PHASE 1

プロトタイプ作成

今日やったこと。Claude Codeでアプリの骨格を作り、動作確認する。この段階で「これは役に立つ」という感触を得ることが目的です。

PHASE 2

コードレビュー

AIが生成したコードの品質をチェックします。セキュリティ上の問題がないか、既存システムとの整合性はどうか。IT部門やエンジニアに見てもらう工程です。

PHASE 3

テスト

想定される使い方でも問題なく動くか確認します。異常なデータを入れたらどうなるか、大量データでも動くかなど、壊れないか検証する工程です。

PHASE 4

社内公開

社内サーバーやAWSにデプロイし、利用者にURLを共有します。マニュアルの用意、問い合わせ先の設定、データバックアップの仕組みも必要です。

Security Tips — 社内公開前のセキュリティレビュー観点

PHASE 2のコードレビューでは、機能面だけでなくセキュリティ面の確認が不可欠です。具体的には、APIキーがコードにハードコードされていないか、ユーザー入力のバリデーション(不正な値の拒否)が実装されているか、アクセス制御(誰が使えるか)が設計されているか、ログ出力に機密情報が含まれていないか。これらの観点はIT部門やセキュリティ担当者と連携して確認してください。

皆さんの強みは「何を作るべきか知っていること」

業務の課題を一番よく知っているのは、技術開発部で日々その業務に向き合っている皆さんです。AI駆動開発によって「課題を知っている人が自分でプロトタイプを作れる」時代になりました。コードレビューやデプロイは専門家に任せても、「何を作るか」「なぜ作るか」を示せることが決定的に大きいのです。

2日間のまとめ

やったことの振り返り

- DAY 1: VSCode + Claude Code の環境構築、カウンターアプリ、チェックリストアプリを開発
- DAY 2: CSVビューア、要件定義ワークショップ、自分の業務アプリの自由開発と成果発表
- デプロイの全体像とTerraformの概要を理解

身につけた3つのスキル

プロンプト設計

AIに伝わる指示の書き方。具体的に、一度にひとつずつ、結果を確認しながら進める原則。

要件定義

業務課題をアプリの要件に変換する力。誰が、何のために、どんな機能を使うかを整理する方法。

段階的开发

MVPから始めて機能を積み上げるアプローチ。完璧を目指さず、まず動くものを手に入れる進め方。

継続学習リソース

Claude Code 公式ドキュメント

VSCode 公式ドキュメント

MDN Web Docs (HTML/CSS/JS学習)

Terraform 入門

AWS はじめてのクラウド

研修後にまず試してほしいこと

今日作ったアプリの改良を続けてください。Claude Code はインストール済みの状態が残っています。業務中に「これ、アプリにできないかな」と思ったら、trainingフォルダで `claude` を起動して試してみてください。使い続けることが、一番の上達法です。